

Página 4 - Quiz de abertura: o que cada linha imprime?

1. Python

```
main.py x + Console
1 print(-2 ** 2, 2 ** 3 ** 2) -4 512
```

2. Java

```
Main.java x + Console
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println(1 + 2 + "3" + 4 + 5);
5     }
6 }
```

3. JavaScript

```
JS index.js x + Console
1 console.log("5" - 2, "5" + 2); 3 52
```

4. C

```
C Main.c x + Console
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

5. Python

```
main.py x + Console
1 x = 0
2 print(x or "padrão", 3 > 2 > 1) padrão True
```

6. JavaScript

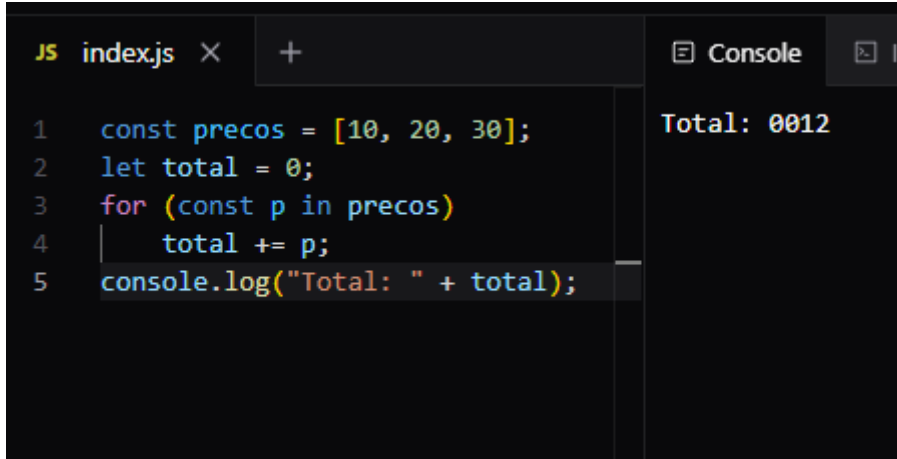
```
JS index.js x + Console I/O
1 console.log(0 == "", "" == "0", 0 == "0"); true false true
```

Resolução de Bugs

Este documento reúne a análise e correção dos 6 exercícios de código apresentados, abordando JavaScript, Python, Java, C e Go.

1. JavaScript

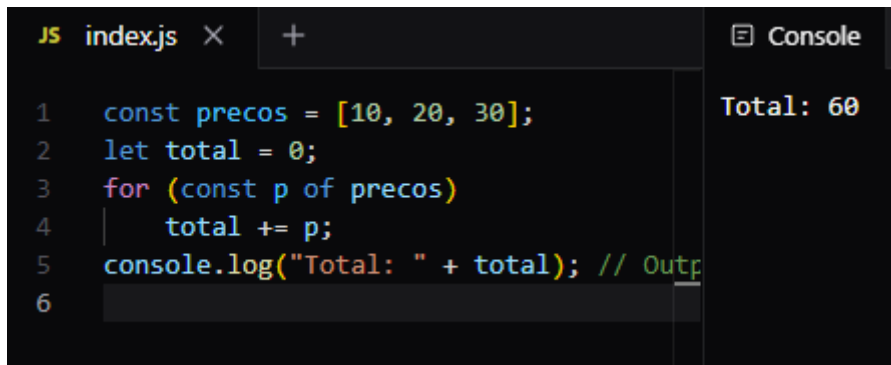
- (a) Saída: Total: 0012



```
JS index.js x + Console
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p in precos)
4   total += p;
5 console.log("Total: " + total);

Total: 0012
```

- (b) Regra que causa o problema: O laço `for...in` itera sobre as **chaves/índices** do array (que são *strings*: "0", "1", "2"), e não sobre os valores. Ao somar `0 + "0" + "1" + "2"`, ocorre a concatenação de strings em vez de soma numérica.
- (c) Como corrigir: Utilize o laço `for...of` para iterar diretamente sobre os valores do array:



```
JS index.js x + Console
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p of precos)
4   total += p;
5 console.log("Total: " + total); // Output
6

Total: 60
```

2. Python

- (a) Saída: 180.0

```
main.py x + Console
1 def desconto(preco, d=None):
2     d = d or 10 # padrão: 10%
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
```

180.0

-
- (b) **Regra que causa o problema:** A expressão `d = d or 10` avalia o valor de `d` em contexto booleano. Em Python, o número 0 é tratado como **falso** (False). Portanto, quando `d=0` é passado, o operador `or` ignora o 0 e atribui o valor padrão 10.
- (c) **Como corrigir:** Verifique explicitamente se `d` é None usando o operador `is`:

```
main.py x + Console
1 def desconto(preco, d=None):
2     if d is None:
3         d = 10 # padrão: 10%
4     return preco * (100 - d) / 100
5
6 print(desconto(200, 0)) # Output: 200.0
```

200.0

3. Java

- (a) **Saída: inválido**

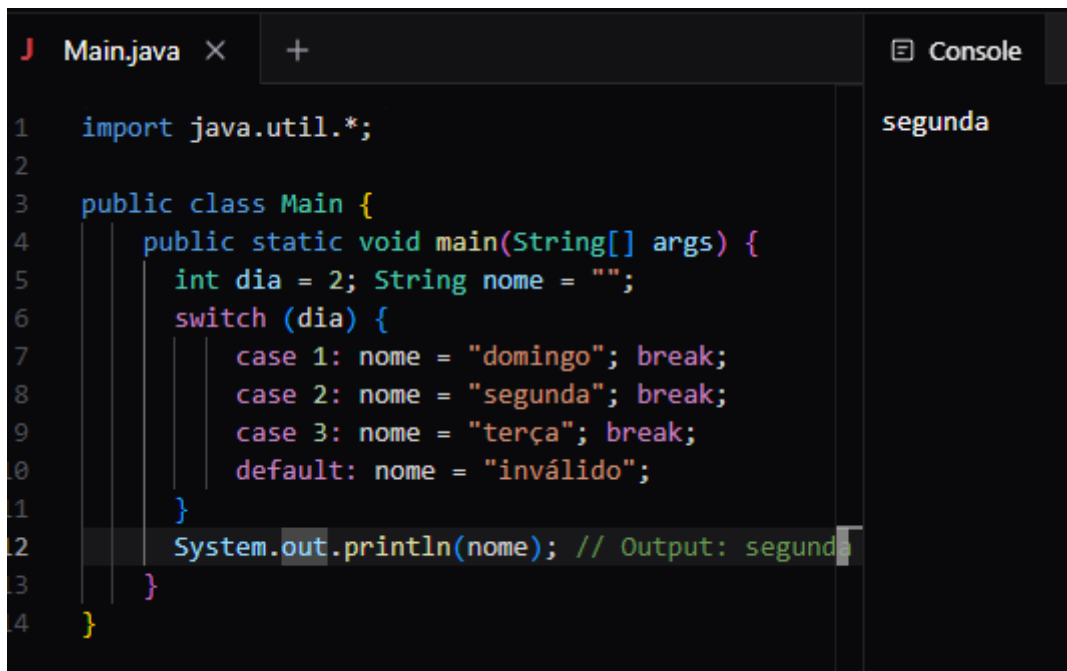
```
Main.java x + Console
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int dia = 2; String nome = "";
6         switch (dia) {
7             case 1: nome = "domingo";
8             case 2: nome = "segunda";
9             case 3: nome = "terça";
10            default: nome = "inválido";
11        }
12        System.out.println(nome);
13    }
14 }
```

inválido

-
- (b) **Regra que causa o problema:** A ausência da instrução `break` ao final de cada case causa o efeito de *fall-through* (queda). O código encontra o case 2, define `nome =`

"segunda", mas continua executando os blocos seguintes (case 3 e default) até o final do switch, sobrescrevendo o valor final com "inválido".

- **(c) Como corrigir:** Adicione a instrução break ao final de cada bloco do switch para interromper a execução:



The screenshot shows a Java IDE with a file named 'Main.java'. The code is as follows:

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int dia = 2; String nome = "";
6         switch (dia) {
7             case 1: nome = "domingo"; break;
8             case 2: nome = "segunda"; break;
9             case 3: nome = "terça"; break;
10            default: nome = "inválido";
11        }
12        System.out.println(nome); // Output: segunda
13    }
14 }
```

The console on the right shows the output: 'segunda'.

4. C

- **(a) Saída:**
saldo insuficiente
fim



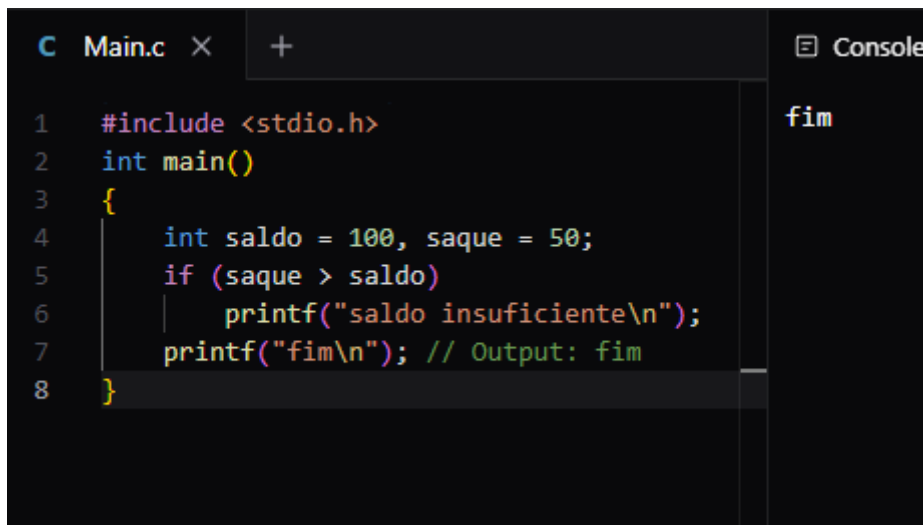
The screenshot shows a C IDE with a file named 'Main.c'. The code is as follows:

```
1 #include <stdio.h>
2 int main()
3 {
4     int saldo = 100, saque = 50;
5     if (saque > saldo);
6         printf("saldo insuficiente\n");
7         printf("fim\n");
8 }
```

The console on the right shows the output: 'saldo insuficiente' and 'fim'.

- **(b) Regra que causa o problema:** O ponto e vírgula ; logo após a condição if (saque > saldo); encerra imediatamente a estrutura do if como uma instrução vazia. Por conta disso, o bloco do printf("saldo insuficiente\n"); deixa de estar condicionado ao if e passa a ser executado incondicionalmente.

- (c) **Como corrigir:** Remova o ponto e vírgula após a instrução if:



```
1  #include <stdio.h>
2  int main()
3  {
4      int saldo = 100, saque = 50;
5      if (saque > saldo)
6          printf("saldo insuficiente\n");
7      printf("fim\n"); // Output: fim
8  }
```

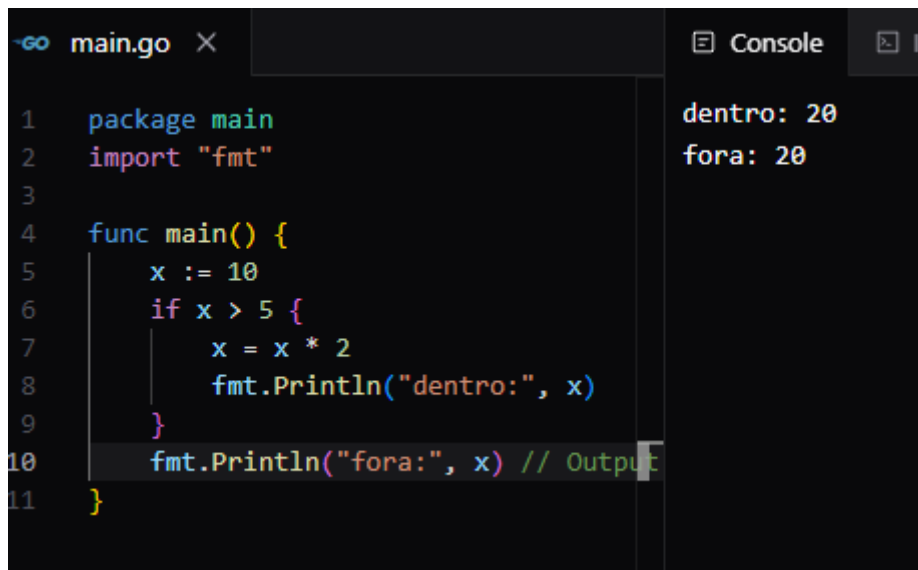
5. Go

- (a) Saída:
dentro: 20
fora: 10



```
1  package main
2  import "fmt"
3
4  func main() {
5      x := 10
6      if x > 5 {
7          x := x * 2
8          fmt.Println("dentro:", x)
9      }
10     fmt.Println("fora:", x)
11 }
```

- (b) **Regra que causa o problema:** O uso do operador de declaração curta := dentro do bloco if cria uma **nova variável local** chamada x (sombreamento de variável ou *variable shadowing*), em vez de reatribuir valor à variável x do escopo externo.
- (c) **Como corrigir:** Utilize o operador de atribuição simples = dentro do if para alterar o valor da variável externa existente:

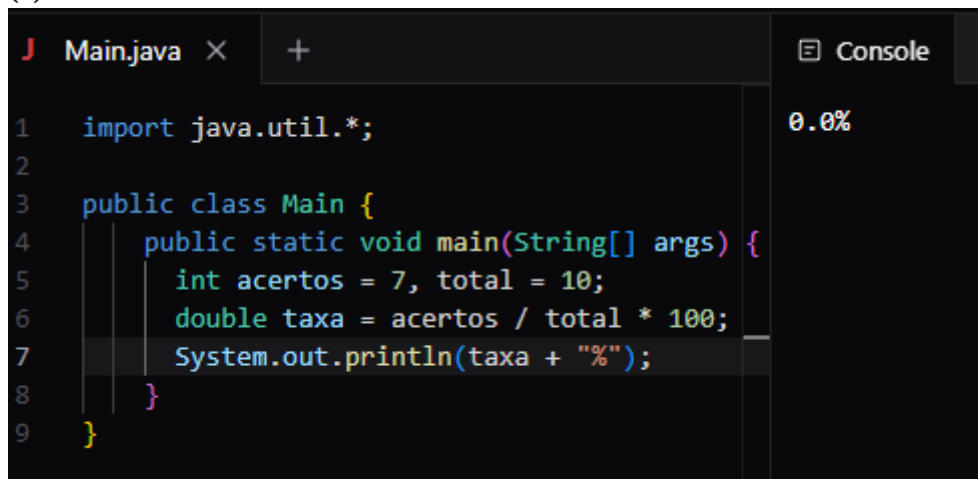


```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6     if x > 5 {
7         x = x * 2
8         fmt.Println("dentro:", x)
9     }
10    fmt.Println("fora:", x) // Output
11 }
```

dentro: 20
fora: 20

6. Java

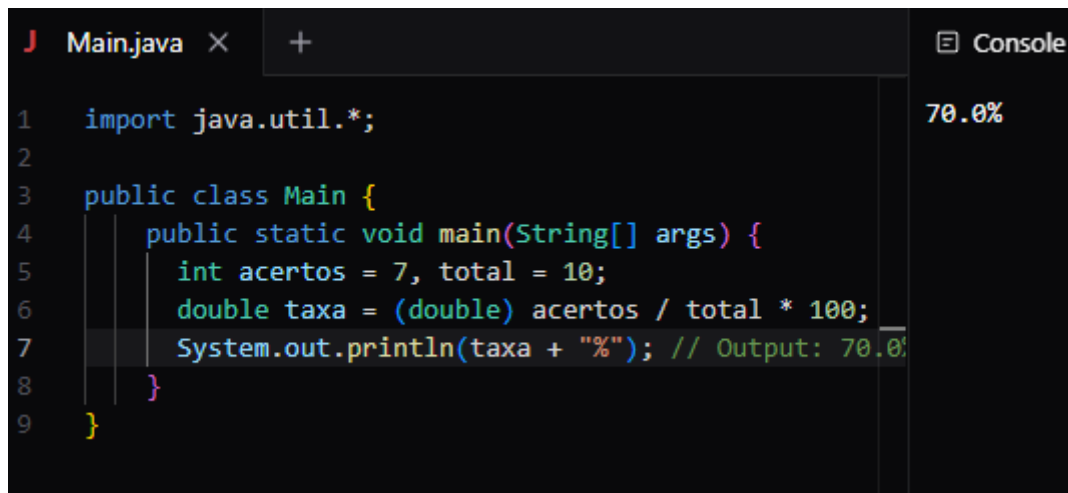
- (a) Saída: 0.0%



```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = acertos / total * 100;
7         System.out.println(taxa + "%");
8     }
9 }
```

0.0%

-
- (b) **Regra que causa o problema:** Como acertos e total são do tipo inteiro (int), a operação `acertos / total` realiza uma **divisão inteira**. O resultado de `7 / 10` é 0 (a parte decimal é descartada) e, em seguida, `0 * 100` resulta em 0, que só então é convertido para double (0.0).
- (c) **Como corrigir:** Faça o *casting* de pelo menos uma das variáveis para double antes de realizar a divisão:



```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          int acertos = 7, total = 10;
6          double taxa = (double) acertos / total * 100;
7          System.out.println(taxa + "%"); // Output: 70.0%
8      }
9  }
```

Página 26 – Atividades complementares (autopesquisa)

Switch em C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(void) {
```

```
    const char *comando = "start";
```

```
    switch (comando[0]) {
```

```
        case 's':
```

```
            if (strcmp(comando, "start") == 0) {
```

```
                printf("Iniciar\n");
```

```
                break;
```

```
            }
```

```
            if (strcmp(comando, "stop") == 0) {
```

```
                printf("Parar\n");
```

```
                break;
```

```

    }

    printf("Comando desconhecido\n");

    break;

case 'p':

    if (strcmp(comando, "pause") == 0) {

        printf("Pausar\n");

        break;

    }

    printf("Comando desconhecido\n");

    break;

default:

    printf("Comando desconhecido\n");

}

return 0;

}

```

Switch em Java

```

public class Main {

    public static void main(String[] args) {

        String comando = "start";

        switch (comando) {

```

```
    case "start":

        System.out.println("Iniciar");

        break;


    case "stop":

        System.out.println("Parar");

        break;


    case "pause":

        System.out.println("Pausar");

        break;


    default:

        System.out.println("Comando desconhecido");

    }

}

}
```

Switch em Go

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    comando := "start"
```

```
switch comando {  
  
case "start":  
  
    fmt.Println("Iniciar")  
  
case "stop":  
  
    fmt.Println("Parar")  
  
case "pause":  
  
    fmt.Println("Pausar")  
  
default:  
  
    fmt.Println("Comando desconhecido")  
  
}  
}
```

Match em Rust

```
fn main() {  
  
    let comando = "start";  
  
  
    match comando {  
  
        "start" => println!("Iniciar"),  
  
        "stop" => println!("Parar"),  
  
        "pause" => println!("Pausar"),  
  
        _ => println!("Comando desconhecido"),  
  
    }  
  
}
```

COMPARAÇÃO ENTRE SWITCH E MATCH

1. Queda entre casos (fall-through)

Uma das principais diferenças entre as linguagens está no comportamento quando um caso é encontrado.

Em C, o switch permite que a execução continue para o próximo caso caso o programador não utilize break. Isso é chamado de fall-through. Dessa forma, quando um caso é executado, os casos seguintes também podem ser executados até que um break seja encontrado.

O Java tradicional possui comportamento semelhante. Um case pode continuar para o próximo caso quando não há um break. Entretanto, versões modernas do Java oferecem uma sintaxe diferente, utilizando setas, que evita esse comportamento e torna a separação entre os casos mais explícita.

No Go, o comportamento padrão é diferente. Cada case termina automaticamente, não sendo necessário utilizar break. Caso seja realmente necessário continuar para o próximo caso, Go fornece a palavra-chave fallthrough.

No Rust, match não possui fall-through. Cada braço do match é independente, e a execução não passa automaticamente para o próximo braço.